

Minicrypt PRFs Do Not Admit Blackbox Oblivious Evaluations

Cruz Barnum
University of Illinois Urbana-Champaign
cruzjb2@illinois.edu

David Heath
University of Illinois Urbana-Champaign
daheath@illinois.edu

Naman Kumar
IRIF & Silence Laboratories
naman.kumar@irif.fr

Mohammad Hajiabadi
University of Waterloo
mdhajiabadi@uwaterloo.ca

Jake Januzelli
Columbia University
jakej@cs.columbia.edu

Mike Rosulek
Oregon State University
rosulekm@eecs.oregonstate.edu

Abstract

Oblivious Pseudorandom Function (OPRF) protocols can be categorized into two variants: *chosen-key* OPRFs – where the server provides the PRF key k as an input – and *ephemeral-key* OPRFs – where the functionality randomly samples k on behalf of the server. Ephemeral-key OPRFs can be constructed from simple cryptography, such as black-box OT and a random oracle. Chosen-key OPRFs, on the other hand, are only known by employing one of the following (more expensive) approaches:

- Express the underlying PRF as a circuit, then use generic MPC techniques to evaluate it in a non-black-box manner.
- Base the underlying PRF on some specific public-key assumption, such as RSA or DDH, then build a custom protocol that achieves obliviousness by leveraging the PRF’s algebraic structure.

Thus, there exists a qualitative gap between known instantiations of these two OPRF variants, in terms of both assumptions and efficiency.

We show that this gap is inherent. In particular, one might hope for an OPRF with all of the following characteristics: the protocol (1) is chosen-key, (2) supports domains of super-polynomial size, and (3) is constructed from “simple” cryptography, such as the minimal assumption of OT, plus a random oracle. We show that no such protocol can exist.

Let Π be any chosen-key OPRF protocol defined in terms of a PRF F , where F is defined with respect to (only) a random oracle. This restriction on F rules out the above approaches of either using a PRF in a non-black-box way or basing the underlying PRF itself on public-key cryptography. While F is restricted in its use of cryptography, the protocol Π is not: Π may use *arbitrary* cryptography, e.g., OT, FHE, iO, etc. We show that each invocation of any such Π *necessarily* leaks information about the server’s key k . After a bounded number of queries, an adversarial client can effectively recover k , breaking server privacy.

To complement our negative result, we provide a matching positive result: we construct a chosen-key OPRF from black-box OT and RO, where server privacy holds for some bounded number of queries n . This protocol’s underlying PRF is constructed from a $(n + 1)$ -wise independent hash function and RO; the server’s key k has length scaling linearly in n which, by our lower bound, is optimal. Thus, our two results tightly (i.e., up to $\text{poly}(\lambda)$ factors) characterize (im)possibility for chosen-key OPRFs, unless one uses non-black-box cryptography or a public-key-style PRF.

1 Introduction

An oblivious pseudorandom function (OPRF) protocol is a two-party protocol allowing a client C with input x to interactively obtain $F(k, x)$, where F is some pseudorandom function family (PRF) and k is a secret key held by a server S [NR04, FIPR05]. OPRF security ensures that C learns nothing about k (beyond that k is consistent with $F(k, x)$), and S learns nothing about x .

OPRF protocols are useful and widely studied objects. They have been prominently used in constructions of private set intersection (PSI) [HL08, JL10, Haz15, KKRT16, KLS⁺17, PRTY19, MPR⁺20] and password-authenticated key exchange (PAKE) [JKK14, JKX18, DHL22, GJK⁺24, KMM⁺25], and they have also found many other uses [FIPR05, JKKX16, JKKX17, BFH⁺20, TCR⁺22], including some in practice [HIJ⁺21]. [CHL22] provides a relatively recent systemization of OPRF techniques.

In this work, we are interested in a gap that has emerged between constructions of two OPRF variants:

- **Chosen-key OPRFs.** The server S passes its key k as input to the functionality. The two parties may repeatedly interact such that after each i -th interaction, the client C obtains $F(k, x_i)$. Chosen-key OPRF protocols include those of [FIPR05, HL08, JL10, JKK14, ECS⁺15, Haz15, JKKX17, CL17, DGS⁺18, Leh19, MPR⁺20, BFH⁺20, DHL22]. We note that [CHL22] refers to chosen-key OPRFs as *multi-point OPRFs*.
- **Ephemeral-key OPRFs.** S 's key k is randomly sampled by the functionality. After each i -th interaction, C obtains $F(k_i, x_i)$ and S obtains k_i . Protocols that achieve this variant include [KKRT16, KMP⁺17, PRTY19].

While the latter variant is sufficient for some applications, especially basic forms of PSI, chosen-key OPRFs are more general and flexible.

Indeed, the chosen-key property is *essential* in certain settings. As a prominent example, chosen-key OPRFs can help solve the *private contact discovery problem* [KLS⁺17], where a client with a small list of contacts would like to compare its list to a large contact database held by a server. This can be solved by using the OPRF to implement PSI over the two sets of contacts. In particular, the server can preprocess $F(k, x_i)$ for each database entry x_i ; then, a client can learn $F(k, y_i)$ for its own contacts by means of calls to an OPRF. With some care, the parties can find their intersecting contacts by comparing their two lists of PRF outputs. For efficiency, the server would like to preprocess its database only once, then re-use the preprocessing across multiple interactions with multiple clients. Clearly, the chosen-key property is essential here: the server re-uses a single key k for all clients. The chosen-key property is also crucial to PAKE, and more generally it increases the flexibility of the OPRF primitive.

Efficiency Gap. So far, ephemeral-key OPRFs have proved easier to instantiate than their chosen-key counterparts. Indeed, several efficient and relatively simple ephemeral-key protocols have been built on top of just black-box oblivious transfer (OT) and a random oracle (RO) [KKRT16, KMP⁺17, PRTY19].

Chosen-key OPRFs, in contrast, have so far required heavier tools. All known chosen-key protocols employ one of the following (more expensive) approaches:

- **Non-black-box cryptography.** Express the underlying PRF as a circuit, then use generic MPC to evaluate the circuit in a non-black-box manner.
- **Public-key assumptions.** Base the underlying PRF on some specific public-key assumption, such as RSA or DDH, then build a custom protocol that achieves obliviousness by leveraging the PRF's algebraic structure.

It is natural to wonder whether the gap in efficiency between these two OPRF variants is inherent. Can we hope for a chosen-key OPRF protocol constructed from the same simple tools as those used to build ephemeral-key protocols?

Construction	Chosen Key	Unstructured F	Black-box F	Query Bound
MPC+PRF	✓	✓	✗	N/A
NR-like [NR04]	✓	✗	✓	N/A
KKRT-like [KKRT16]	✗	✓	✓	1
Non-colluding Clients [WLLM25]	✓	✓	✓	1 per client
Impossible!	✓	✓	✓	N/A
Ours	✓	✓	✓	n

Figure 1: Known OPRFs either (1) use non-black-box cryptography, e.g., use MPC to evaluate the PRF F , (2) follow the paradigm of [NR04] and define F with respect to some structured assumption (DDH, RSA, LWE, etc.), (3) are ephemeral-key protocols and hence support only one query, or (4) are chosen-key, but support only one per-client query [WLLM25]. We show that the most desirable form of OPRF – where unstructured F may be evaluated in a black-box manner any (polynomial) number of times – is impossible. We also show it *is* possible to get this desirable form of OPRF, if an explicit query bound n is chosen *a priori*.

1.1 Contribution

The existing gap between chosen-key and ephemeral-key OPRF protocols is inherent. We show that it is impossible to construct a chosen-key OPRF protocol without either (1) using non-black-box cryptography, or (2) basing the underlying PRF F on some structured hardness assumption.

More precisely, consider a PRF F that derives its cryptographic hardness from a random oracle (RO) alone. This restriction captures the idea that the two above strategies are not used. Let Π be a chosen-key OPRF protocol that securely computes F . Note that while we insist F is based entirely on RO, we place *no* restrictions on the protocol Π , so Π can use arbitrarily powerful cryptography. We show that Π *necessarily* leaks to an adversarial client information about the server’s key k . The consequence is that security against an adversarial client can only possibly hold for some bounded number of interactive evaluations, where the bound depends on the length¹ of the server’s key.

Following is an informal statement of our main lower bound.

Theorem 1 ((Informal) Interactive Evaluation Bounds on OPRF Schemes). *Let Π be an OPRF scheme from a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles PKO , which are independent from $\mathcal{O}$. Suppose the underlying PRF F is a poly-sized circuit with oracle access (only) to $\mathcal{O}$. Let the number of oracle queries that F and the server make during a single interactive evaluation be bounded by polynomial $p(\lambda)$. Suppose that the key length is bounded by polynomial $p(\lambda)$.*

Then for a sufficiently large, polynomial-sized domain F , there exist constant advantage bounds such that it is not possible for Π to simultaneously satisfy correctness, client privacy, and $\omega(\lambda^2 \cdot p(\lambda)^2)$ -one-more security with the aforementioned constant advantage bounds.

Our impossibility result implies difficulty for certain applications of OPRFs. For instance, there exists a corresponding efficiency gap between PSI protocols where the protocol is “one shot” (e.g., as solved by the [KKRT16] approach and its follow-on works) versus one where the server *preprocesses* its set and then re-uses that preprocessing with multiple clients (for example as described above for contact discovery). Our impossibility result implies that multi-client PSI cannot compete with the performance of single-client PSI, at least not by using the most obvious approach of using chosen-key OPRFs.

Our considered notion of security. The literature has considered several notions of OPRF security (see Section 2.3 and Appendix D), and one might wonder if our impossibility result applies only in the context of

¹There are subtleties in even defining the server’s key length, as the server might, e.g., derive a long PRF key from a short seed. Our proof accounts for this subtlety.

a particularly strong security notion. Our impossibility result holds even when one considers extremely weak notions of OPRF security, and thus the impossibility holds for a broad class of possible OPRF definitions.

Specifically, our attack works even when (1) the PRF F maps a domain of some polynomial size to a single bit, (2) the OPRF protocol is semi-honest, (3) the adversarial client is restricted to making uniformly chosen OPRF queries, and (4) after seeing the output of such queries, the adversarial client must guess the output of a uniformly chosen query from unseen queries. This security notion is weaker than standard definitions in the literature, including one-more pseudorandomness and one-more unpredictability [JL10, ECS⁺15, Leh19], which relate to distinguishing (resp. guessing) the output of a challenge query from uniform (resp. function output) given prior interactive evaluations, which could leak information about the key.

Positive Results. Our impossibility result asserts that a black-box chosen-key OPRF inherently leaks *some* information about the server’s key. Thus, it remains plausible that one *can* use RO and OT to construct a chosen-key OPRF that remains secure for a bounded number of queries. Indeed, to complement our negative result, Appendix E provides security definitions and a simple OPRF construction. Our construction achieves a particularly strong notion of simulation-based server privacy, so long as the client issues only an *a priori* bounded number of evaluations. Our construction of F is secure for n queries, and it is based on an $(n + 1)$ -wise independent hash function and a random oracle. Even after the adversary sees n OPRF interactions, the underlying (non-oblivious) PRF $F(k, \cdot)$ remains secure in the standard sense: outputs of the keyed PRF $F(k, \cdot)$ are pseudorandom.

Our upper bound helps characterize the relative strength of our impossibility result. Namely, our impossibility result implies that the best OT-based OPRF we can hope for is one supporting a bounded number of queries; our upper bound confirms that this weakening is sufficient. Figure 1 summarizes.

Our construction itself can be instantiated from any MPC protocol capable of evaluating an n -wise independent hash function, and hence our construction can be efficiently constructed assuming only OT.

Because our impossibility result rules out a broad class of OPRF constructions, we feel the notion of query-bounded OPRFs – which circumvent our impossibility – may be interesting as a security goal for future positive results.

1.2 Related Work and Motivations for Studying Impossibility

OPRFs with support for bounded numbers of queries. Pinkas, Schneider, and Zohner [PSZ14] first constructed ephemeral-key OPRF protocols based on oblivious transfer, in the setting of PSI. Their work prompted several followups [PSSZ15, KKRT16, BC23] that further refine the approach, all of them building on the celebrated IKNP OT extension [IKNP03]. These works all produce a batch of OPRF evaluations of the form $F(k_i, x_i)$, for n distinct ephemeral server keys k_i and n client inputs x_i . Subsequent followup works [PRTY19, CM20, PRTY20, GPR⁺21, RS21] realized an efficient variant with just a single OPRF key for all client inputs x_i , but the key is still ephemeral. After evaluating the OPRF on a fixed batch of client inputs, the key cannot be reused for subsequent OPRF instances.

It seemed natural to try to port these OT-based OPRF techniques to the “multi-client” PSI setting, where a server computes a single encoding of its set and then can safely re-use that encoding across multiple protocol invocations. Such an upgrade would essentially entail upgrading the underlying OPRF from an ephemeral-key OPRF to a chosen-key one. Our results explain why no such upgrade has been identified. Indeed, one motivation underlying this paper was to understand why OT-based PSI seems insufficient to achieve a multi-client variant of PSI.

Post-quantum OPRFs. Many OPRF protocols are based on the hardness of number-theoretic assumptions, e.g. DDH or RSA. It is well known that such assumptions are false in the presence of a quantum adversary, and hence such OPRF protocols would be insecure. While many plausibly-post-quantum OPRFs have been constructed, e.g., [BKW20, ADDS21, Bas22, YBH⁺25], they are relatively inefficient when compared with their number-theoretic counterparts.

Another route to plausibly achieve a post-quantum OPRF is to simply evaluate a post-quantum-secure PRF via a post-quantum MPC protocol. Indeed, several OT protocols have been proposed for the post-quantum setting [MR19, DGH⁺20, DCZ⁺24], and efficient OT extension protocols are constructed either in Minicrypt or from LPN [IKNP03, BCG⁺19, Roy22], both of which appear to resist quantum adversaries. Hence, the post-quantum setting motivates our study of the difficulty of constructing efficient OPRFs from OT alone.

Unfortunately, our results suggest that post-quantum-secure chosen-key OPRFs will need to either (1) use non-black-box cryptography, (2) base the underlying PRF F on some structured plausibly-post-quantum-secure hardness assumption, or (3) support only a bounded number of queries.

OPRFs with Batched Input/Output. Our impossibility result shows that information about the server’s effective key is necessarily leaked in proportion to the number of protocol invocations. However, some OPRF protocols, e.g., [PRTY19], compute multiple outputs $F(k, x_i)$ under the same key k in a *single invocation*. We refer to these as *batched* OPRF protocols.

Since our impossibility result considers only the number of protocol invocations, it is possible that considering batching may help to partially circumvent difficulty of constructing simple OT-based OPRFs, in the sense that it might be possible to use a small amount of server key material to evaluate a large number of OPRF queries. Note that our impossibility result still means that the number of batched protocol invocations must be bounded.

Other notions of OPRFs. Recent work [WLLM25] proposed a chosen-key OPRF in Minicrypt, which at the surface appears to contradict our main theorem. However, their construction does not permit the server to reuse the same chosen key across different protocol instances. Their construction is useful in a restrictive setting where (1) each client invokes the OPRF at most once and (2) no two clients collude. Our impossibility result is considered in the context of an OPRF satisfying a more standard notion of security where a single adversary may query the OPRF repeatedly. As witnessed by our impossibility result, the [WLLM25] construction cannot satisfy such a security notion.

1.3 Proof Intuition

Recall that our model insists that the underlying PRF F is defined *only* in terms of a random oracle (RO). Since RO is the only source of assumed hardness, F *must* make non-trivial RO queries. Indeed, as is standard in such lower bounds, we model the adversary as having access to a PSPACE oracle that allows it to essentially compute anything not explicitly externalized by some oracle, so RO queries are necessary to achieve PRF security.

Therefore, the combined RO queries issued by the client and the server *must* be sufficient to successfully evaluate F . This must be true regardless of what other cryptographic resources (also modeled as oracles) are available to the OPRF protocol Π , as otherwise Π will be *incorrect*.

In short, this simple observation – that at least one of the two parties *must* issue each RO query made by F – ultimately leads to our impossibility result. Very roughly, and as we explain in much more detail next, this is problematic for the following reasons:

- If F issues a particular RO query that depends on the server’s key k and then the client C makes that query, then that conveys to C information about k , which is problematic for server privacy.
- If F issues a particular RO query that depends on the client’s input x and then the server S makes that query, then that conveys to S information about the client’s input x , which is problematic for client privacy.
- If F issues no RO query depending on both k and x , then that is problematic for the PRF security of F .

Thus, in some high-level sense, it *must be* that Π fails to achieve oblivious evaluation. Of course, the OPRF protocol Π might choose F carefully, or carefully split RO queries between client and server, with the goal of leaking as little about k (resp. x) as possible. Thus, showing impossibility requires a significantly more careful accounting than the above bare-bones sketch. The rest of this section explains at a high level. Note that relevant sections will be referenced in the paragraph headers.

Simple Model Attack (Section 3.3). For simplicity, first consider an OPRF protocol where the server makes *no* queries to the random oracle. Later we will explain how the ideas from this simple setting generalize. Because the server does not issue RO queries, the client must make all RO queries. Let us suppose the client is corrupted.

Now, suppose that we – the adversarial client – maintain a list of all possible keys $\mathcal{K}$ that the server could be holding, which we hope to whittle down as we issue interactive OPRF queries; we will handle the fact that $\mathcal{K}$ could be exponential in size later. In particular, suppose that we set some arbitrary constant s and then split the keys in $\mathcal{K}$ into two groups: *consistent keys* and *inconsistent keys*. Let k^* be the server’s key. We say that a key k is *s-consistent* or simply *consistent* if

$$F^{\mathcal{O}}(k, \cdot) = F^{\mathcal{O}}(k^*, \cdot)$$

on at least an s portion of inputs; if a key k is not consistent, it is *inconsistent*.

Suppose we are somehow able to remove *all* inconsistent keys from $\mathcal{K}$, yielding a much smaller set of possible keys $\mathcal{K}'$. In this case, we can choose any key $k \in \mathcal{K}'$ and break server privacy by determining the value of $F^{\mathcal{O}}(k^*, x^*)$ for a uniform, previously unqueried input x^* . The attack does this by simply evaluating $F^{\mathcal{O}}(k, x^*)$, which, by definition, should equal $F^{\mathcal{O}}(k^*, x^*)$ with probability close to s .

However, this attack required that we successfully remove inconsistent keys from $\mathcal{K}$. To do so, we propose an attack in which we iteratively issue interactive OPRF queries while maintaining the following two properties:

- With high probability, the server’s key k^* is not ruled out from $\mathcal{K}$.
- Each inconsistent key k is ruled out from $\mathcal{K}$ with some constant probability.

Note that it is at this point that our attack relies critically on the protocol being chosen-key: we assume that the key k^* remains fixed across multiple interactions.

On each i -th iteration, we refine our set of possible keys $\mathcal{K}_i$. Then after some number n of iterations, we can guarantee that $\mathcal{K}_n$ contains at least one element – i.e. k^* – and that it contains *no* inconsistent keys, except with negligible probability. On each i -th iteration, we can refine $\mathcal{K}_i$ by observing that – due to new information observed during the interaction – certain keys have become inconsistent. *The cornerstone of our attack* is that it is sufficient to determine whether or not a given key is consistent.

Checking Key Consistency. Let ε_c denote the correctness error of the OPRF protocol. Because we are given a PSPACE oracle, there is an algorithm, which we call **Check**, which when given F , an input x , a candidate key k , and a set of RO queries Q , can either (1) determine the PRF output $F^{\mathcal{O}}(k, x)$ or (2) find that, given only Q , $F^{\mathcal{O}}(k, x)$ cannot be determined with probability higher than $1 - \varepsilon_c$. Because we have temporarily assumed that the server cannot make RO queries, this means that the adversarial client can locally determine whether a given candidate key k could have possibly yielded the observed OPRF outputs. If a candidate key k frequently disagrees with the observed OPRF output, or if the output of F could not be computed correctly with high probability, then we can conclude that k is likely inconsistent and remove it from $\mathcal{K}$.

Handling Imperfect Correctness (Section 3.4). To ensure that we do not accidentally rule out the server’s key k^* (due to some inherent correctness error of the OPRF protocol), we can instead only rule out keys that differ from the observed output on a large portion, say $1/3$, of some large batch of trials, say λ trials.

Handling Exponentially-Sized $\mathcal{K}$. In our actual attack, we will not be maintain $\mathcal{K}_i$ concretely, as these sets are too large. Instead, we use the PSPACE oracle to maintain a virtual set of keys. In particular, we maintain a list of the consistency checks that we have done so far; this list is poly-size by construction. Then, after completing the attack, we can query the PSPACE oracle for the first key that satisfies all consistency checks simultaneously. This has the same effect as maintaining the key set up to $\mathcal{K}_n$ and then sampling the lexicographically smallest key from $\mathcal{K}_n$.

Handling Extension Queries (Section 3.5). So far, we have simplified discussion by supposing that the server makes no RO queries. Of course, in designing an OPRF protocol, it may be useful to let the server issue such queries, so our proof must account for them.

In our attack, we categorize server queries into two types: *key extension queries*, and *infrequent queries*. Key extension queries are simply those queries that the server or F make with high probability w.r.t. a given key k . These queries are likely to be made independently of the client's input. Roughly, such queries have the effect of extending the server's key. For example, by using the RO as a generator, the server can arbitrarily stretch a short key, but in doing so will it will have to make those RO queries each time the key is used. Indeed, our attack treats such queries as part of the server's key. Because the server can only make poly-many RO queries, the effective length of the key remains bounded. Our attack shows that our considered class of OPRF protocols can only maintain security for a number of interactive evaluations scaling with the key's *effective* length.

Handling Infrequent Queries (Section 3.6). The second category of server RO queries, infrequent queries, are simply queries that the server and F make with low probability (where probability is over x), again with respect to some particular key k . We break this category down further into infrequent *material* and infrequent *immaterial* queries. Material queries are those queries that, for some client input x , some server key k^* , and random oracle $\mathcal{O}$, are the RO queries that F makes in $F^{\mathcal{O}}(k^*, x)$. Immaterial queries are simply queries that are not material.

As described above, our attack relies heavily on the ability to predict the probability that the protocol as a whole could correctly produce $F^{\mathcal{O}}(k^*, x)$ with just the random oracle queries that the entire protocol (i.e., both client and server) make. If the server is making queries that the client is not privy to, then it could be the case that the client's prediction of maximum protocol correctness is an underestimate, which has the effect potentially that the client concludes that the server's key k^* is not a candidate key since the client erroneously concludes that the RO queries he is asked to make are routinely insufficient to compute the outcome of F to a high enough degree.

It turns out that server queries do not significantly help the protocol compute the output more than the client's queries. To formalize this, we consider a sister algorithm to **Check**, called **Find**, which given queries Q , server key k^* , and x finds the probability with which the highest probability outcome of F can be determined given the set of made queries. In general, the protocol cannot hope to find the outcome better than **Find** says, so we will bound the difference between the probability **Find** gives between queries that the server and client make in union and those queries that only the client makes.

The intuition behind bounding infrequent material queries is that by the definition of infrequency, the server is only making queries with some probability, say s . Then even though material queries might increase the protocol's probability of computing a correct output arbitrarily (maximum w.p. 1), there can only be a maximum of $p_F(\lambda)$ such queries at any given time.

For infrequent immaterial queries, we find that by the definition of infrequent, such queries are made by F with probability s , say. Because these queries are not material, they cannot help to directly compute the output of F , but instead may give insight into the likely outcome of F in the event that the protocol cannot compute F directly. By only being made w.p. s by F though, we find that **Find** only attributes at most a weight of s to each such query. As such, each query only gives an expected difference of at most s in the outcome of **Find**.

Since the server can only make $p_S(\lambda)$ RO queries, we find that the expected difference is $s \cdot p_S(\lambda)$, which means that we can set the expectation to whatever constant we would like by setting s appropriately. After

doing this, we can use a Markov Inequality to bound the two cases where the difference is under some threshold, which if small enough we can eat as a slight difference in rejection probability for the server's key k^* , or if over when can accept this as more error, again increasing our rejection probability of k^* by some small amount.

Using Client Privacy. The above arguments rely on the inability of the server to make material queries, or in general to sway her distribution over queries according to the client's input. The above arguments work assuming perfect client privacy, in which case the distribution of the server's RO queries can only depend on her key k^* . To upgrade the result to allow for constant advantage ε_s w.r.t. client privacy, we can model the advantage as some statistical difference between the distribution of queries that the server would have made given a uniform client input, and those for any given client input. Then, because this error is simply the sum of differences between queries in both cases, we find that the logic from the previous paragraph works with an extra cumulative ε_s term across all of the queries, which pops out due to the linearity of expectation.

Thus, we can simply set ε_s to be sufficiently small and recover our prior logic with an extra slight error term.

Intuition Caveats. The formal proofs in Section 3 are structured similarly to above section, but there are many small details that were left out of the intuition, such as how to handle small-domain PRFs, or how to bound the probability that the server's key k^* is ruled out. The latter detail is slightly more complicated than as described above, since the protocol may only provide enough oracle queries to evaluate F with probability slightly less than the correctness bound ε_c , except on some small – but large enough – portion of inputs to achieve the promised correctness.

Giving explicit parameters for error in terms of correctness and security makes the analysis complicated, but if those terms are negligible, then the above intuition is a good one. For example, having a negligible error makes the probability of ruling out k^* due to error negligible in any given trial, which would greatly simplify the analysis.

2 Preliminaries

2.1 Notation

Notation 1 (General Notation).

- $x, y, \tau \leftarrow \langle A, B \rangle$ denotes a protocol execution between interactive algorithms A and B ; x denotes the output of A , y the output of B , and τ the protocol transcript.
- $_ \leftarrow A(x)$ denotes that the output of $A(x)$ can be ignored, so is not assigned a variable for clarity.
- $\text{Dummy}(\tau)$ denotes an interactive protocol that plays the role of the right party according to the transcript τ , for the purposes of recounting a run of an interactive protocol.
- $X \approx Y$ denotes that probability ensembles X and Y are computationally indistinguishable (in λ).
- $x \leftarrow \$ A$ denotes sampling a variable x uniformly from a set A .
- $r \leftarrow \$ \{0, 1\}^*$ denote a random tape being sampled to a sufficient length for some PPT algorithm. Such r has polynomial length.

Notation 2 (PSPACE Constraint). A PSPACE **constraint** is a description of a predicate given by an algorithm, which potentially may need PSPACE to be evaluated. Code associated with a constraint will be denoted with quotations.

We also provide a list of symbols that we use in proofs in this work, found in Appendix A. If one forgets what a symbol is referring to in the middle of a proof, one can reference the list in Appendix A.

2.2 n -wise Independent Hash Families

Our upper bound is defined in terms of a n -wise independent hash family; we recall the well-known notion:

Definition 1 (n -wise Independent Hash Family). $\mathcal{H} \subseteq \mathcal{X} \rightarrow \mathcal{Y}$ is a **n -wise independent hash family** if for all distinct inputs $x_1, \dots, x_n \in \mathcal{X}$ and all (not necessarily distinct) outputs $y_1, \dots, y_n \in \mathcal{Y}$, we have

$$\Pr[h(x_1) = y_1 \wedge \dots \wedge h(x_n) = y_n \text{ where } h \leftarrow \mathcal{H}] = \frac{1}{|\mathcal{Y}|^n}.$$

It is well known that n -wise independent hash families exist. For example, a standard approach based on random polynomials achieves n -wise independence, where $\mathcal{X} = \mathcal{Y} = \mathbb{F}$ for finite field $\mathbb{F}$, and where seeds consist of n elements of $\mathbb{F}$.

2.3 OPRF Definitions

We give a definition of Oblivious PRF correctness as adapted from [FIPR05]. Note that some interpretations of correctness may require that an evaluation occurs correctly with high probability w.r.t. *every* key k that could be sampled; however, we present a weaker definition for the benefit of our lower bound.

Definition 2 (Chosen-Key Oblivious Pseudorandom Function). An **oblivious pseudorandom function (OPRF)** Π is a four-tuple of algorithms (Gen, C, S, F) :

- $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is a (non-oblivious) pseudorandom function family.
- Gen takes as input a security parameter 1^λ and outputs a PRF key $k \in \mathcal{K}$.
- C is an interactive algorithm to be run by the client. When C interacts with the server program (see next) on input x , it outputs a value $y \in \mathcal{Y}$.
- S is an interactive algorithm to be run by the server. S takes as input a key.

Π is **correct** if the following probability is overwhelming (in λ):

$$\Pr \left[F(k, x) = y \text{ where } \begin{array}{l} k \leftarrow \mathcal{K} \\ x \leftarrow \mathcal{X} \\ y, \neg, \tau \leftarrow \langle C(1^\lambda, x), S(k) \rangle \end{array} \right] > 1 - \text{negl}(\lambda)$$

2.4 Our Considered Notions of OPRF Security

The definitions in this section are not intended to be used for constructive results. Instead we show that OPRF schemes that satisfy even these extremely weak security definitions still must abide by our lower bound.

We give a significant weakening of client privacy, in which a client's input is only indistinguishable w.r.t. a uniform input or another fixed input. This definition is quite weak in that it specifies that client privacy needs only hold for a *single* OPRF evaluation:

Definition 3 (One-Time Indistinguishability-Based Client Privacy). An OPRF scheme $\Pi = (F, Gen, C, S)$ is **one-time indistinguishability-based client private** if for all keys $k \in \text{Supp}(Gen(\lambda))$, and all inputs $x \in \mathcal{X}$, the following ensembles are indistinguishable:

Uniform _{Π, k} ^{ear} (λ)		Fixed _{Π, k, x} ^{ear} (λ)
1 : $x^* \leftarrow \mathcal{X}$		1 : $x^* \leftarrow x$
2 : $r \leftarrow \{0, 1\}^*$	$\approx$	2 : $r \leftarrow \{0, 1\}^*$
3 : $\neg, \neg, \tau \leftarrow \langle C(1^\lambda, x^*), S(k; r) \rangle$		3 : $\neg, \neg, \tau \leftarrow \langle C(1^\lambda, x^*), S(k; r) \rangle$
4 : return (r, τ)		4 : return (r, τ)

On the server side, we posit a weak version of the more standard notion of *n-one-more security* (Appendix D, Definition 7) for a weak PRF. The definition states that even after the client observes n OPRF evaluations on uniform inputs, a single extra (non-oblivious) PRF output should appear random:

Definition 4 (*n*-Oblivious Weak PRF One-More Security). *An OPRF scheme Π is **n-weak PRF one-more secure** if $|\mathcal{X}| > n$, and for the following game*

$\text{Game}_{\Pi,n,b}^{\text{wom}}(\lambda)$	
1 :	$k \leftarrow \text{Gen}(1^\lambda)$
2 :	$\mathcal{X}' \leftarrow \emptyset$
3 :	for $i \in [n]$:
4 :	$x_i \leftarrow \$ \mathcal{X}$
5 :	$\mathcal{X}' \leftarrow \mathcal{X}' \cup \{x_i\}$
6 :	$r_i \leftarrow \$ \{0, 1\}^*$
7 :	$y_i, \tau_i \leftarrow \langle C(1^\lambda, x_i; r_i), S(k) \rangle$
8 :	$x^* \leftarrow \$ \mathcal{X} \setminus \mathcal{X}'$
9 :	$y_0^* \leftarrow F(k, x^*)$
10 :	$y_1^* \leftarrow \$ \mathcal{Y}$
11 :	return $(x^*, y_b^*, (x_1, y_1, r_1, \tau_1), \dots, (x_n, y_n, r_n, \tau_n))$

Then

$$\{\text{Game}_{\Pi,n,0}^{\text{wom}}(\lambda)\} \approx \{\text{Game}_{\Pi,n,1}^{\text{wom}}(\lambda)\}.$$

3 Lower Bound

We will illustrate the techniques to achieve an impossibility result in a simplified model, and then show how to extend the results of the simple model to the general case.

3.1 The General Model

Our goal is to show the following formal result.

Theorem 1 (Interaction Bounds on RO-based OPRFs). *Let $\Pi = (\text{Gen}, C, S, F)$ be an efficient OPRF scheme from a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles $\text{PK}\mathcal{O}$, which are independent from $\mathcal{O}$. Also require that F is a poly-sized circuit with only oracle access to $\mathcal{O}$. Say that the number of random oracle queries that F and S make during an execution is bounded by polynomials $p_F(\lambda)$ and $p_S(\lambda)$ respectively. Let $\eta < 5/12 - 2^{-\Omega(\lambda)}$ be a constant. Say that the key length is bounded by $\ell(\lambda)$ and that*

$$|\mathcal{X}| \geq (\lambda \cdot \ell(\lambda) + 36864\lambda^2 \cdot (p_S(\lambda) + p_F(\lambda))^2 + \lambda)/\eta.$$

Then it is not possible for Π to simultaneously satisfy correctness and one-time indistinguishability-based client privacy (Definitions 2 and 3) and

$$(\lambda \cdot \ell(\lambda) + 36864\lambda^2 \cdot (p_S(\lambda) + p_F(\lambda))^2 + \lambda)$$

-weak one-more security (Definition 4) with constant bounds (in place of $\text{negl}(\lambda)$) $\varepsilon_c \leq 1/96$, $\varepsilon_s \leq 1/9216$, and $\varepsilon_i \leq (5/12 - \eta)/4 - 2^{-\Omega(\lambda)}$ respectively.

3.2 PSPACE Algorithms

We begin by giving an algorithm for checking the consistency of a set of random oracle queries against a run of the underlying PRF $F^\mathcal{O}(k, x)$ for some key k and input x . In particular, we would like to determine if the given set of oracle queries Q are sufficient to compute the output of $F^\mathcal{O}(k, x)$ to some degree of correctness. We will use the consistency checker to bound the maximum correctness that a given system (client and server) can achieve w.r.t. some target computation $F^\mathcal{O}(k, x)$.

We defer the constructions of the following lemmas to Appendix B, since they are quite straight forward given a PSPACE oracle.

Lemma 1 (Consistency Checker). *Let $F^\mathcal{O}(\cdot, \cdot)$ be a circuit with oracle access. There exists a PSPACE algorithm **Check** that on input (1) the description of F , (2) a set of queries $Q = \{(q, o) \mid \mathcal{O}(q) = o\}$, (3) inputs k and x , and (4) a parameter ε_c , returns one of the following:*

- *The unique value y s.t., given the oracle queries Q , it is the case that $F^\mathcal{O}(k, x) = y$ with probability at least $1 - \varepsilon_c$. (Note that this requires $\varepsilon_c < 1/2$ to guarantee uniqueness.)*
- *The symbol NO, if there is no such y .*

Then by binary search, or modifying the algorithm of the above slightly,

Corollary 1 (Consistency Finder). *Let $F^\mathcal{O}(\cdot, \cdot)$ be a circuit with oracle access. There exists a PSPACE algorithm **Find** that on input (1) the description of F , (2) a set of queries $Q = \{(q, o) \mid \mathcal{O}(q) = o\}$, and (3) inputs k and x , returns (y, p) , where y is the most probable element to be output by F – random element if tied – given oracle queries Q , and p is the probability with which y is returned.*

Lemma 2 (Constraint Sample). *Let Γ be a set of PSPACE constraints over the set $\{0, 1\}^{\ell(\lambda)}$ for some implicit $\ell(\lambda)$. Then there exists a PSPACE algorithm **ConstraintSample** that returns one of the following:*

- *The smallest string k (when interpreted as an $\ell(\lambda)$ -bit integer) that satisfies all of the constraints in Γ .*
- *$\perp$ if no such $k \in \{0, 1\}^{\ell(\lambda)}$ exists.*

3.3 Simplified Model

As a warm-up, we will consider a simplified OPRF model. This model will place two constraints on the OPRF protocol that greatly simplify analysis:

- The OPRF scheme must be *perfectly correct*, i.e. the client must output $F^\mathcal{O}(k, x)$ with probability 1.
- The honest OPRF server must make no queries to the random oracle $\mathcal{O}$.

We will remove these constraints later.

Lemma 3 (Interactive Evaluation Bounds for OPRF Schemes in the Simplified Model). *Let $\Pi = (Gen, C, S, F)$ be an efficient OPRF scheme defined in terms of a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles $PK\mathcal{O}$, which are independent of $\mathcal{O}$. Also require that F is a poly-sized circuit with only oracle access to $\mathcal{O}$ and S **cannot make oracle queries**. Let $0 < \eta < 1$ be a constant. Say that the key length is bounded by $\ell(\lambda)$ and that*

$$|\mathcal{X}| \geq (\ell(\lambda) + 1) \log_{2\varepsilon_i + \eta}(1/2)/\eta$$

for ε_i defined below.

*Then it is not possible for Π to simultaneously satisfy **perfect** correctness (Definition 2) and*

$$\lceil (\ell(\lambda) + 1) \log_{2\varepsilon_i + \eta}(1/2) \rceil$$

-weak one-more security (Definition 4) with constant bounds (in place of $\text{negl}(\lambda)$) $\varepsilon_c = 0$, and $\varepsilon_i < (1 - \eta)/2$ respectively.

Proof.

The attack we propose will record random oracle queries and outputs from running the interactive evaluation for each x_i given² and add a constraint that prospective keys should agree with the output y_i when given x_i . After doing this sufficiently many times, we guarantee with high probability that the remaining keys agree with the server's key k^* on most points, and so the attack samples any key that satisfy every constraint and uses this to predict the output bit y^* .

The Adversarial Client. We give the candidate distinguisher against the oblivious weak PRF one-more game of Definition 4. For simplicity, assume that F outputs a single bit.

```

 $\mathcal{D}^{\mathcal{O}, \text{PSPACE}, \text{PKO}}(1^\lambda, x^*, y^*, (x_1, y_1, r_1, \tau_1), \dots, (x_n, y_n, r_n, \tau_n))$ 
1 :  $\Gamma \leftarrow \{ "k \in \text{Supp}(\text{Gen}(1^\lambda)) " \}$ 
2 : for  $i \in [n]$  :
3 :   Record  $\mathcal{O}$  queries  $Q_i$  from  $y_i, -, - \leftarrow \langle C^{\mathcal{O}, \text{PSPACE}, \text{PKO}}(1^\lambda, x_i; r_i), \text{Dummy}(\tau_i) \rangle$ 
4 :    $\Gamma \leftarrow \Gamma \cup \{ \text{"Check"}^{\text{PSPACE}}(F, Q_i, k, x_i, 0) = y_i " \}$ 
5 :  $k \leftarrow \text{ConstraintSample}^{\text{PSPACE}}(\Gamma)$ 
6 :  $b' \leftarrow F^{\mathcal{O}}(k, x^*)$ 
7 : return  $b' \oplus y^*$ 

```

Recall that **Check** and **ConstraintSample** are given in Lemma 1 and Lemma 2 respectively, and that **Dummy** is an interactive protocol that just echos the transcript back so that we can go through the transcript from the client's perspective. Further, Γ is a set of constraints over the set $\{0, 1\}^{\ell(\lambda)}$ where the constraints are implicitly over keys $k \in \{0, 1\}^{\ell(\lambda)}$, i.e. any time k appears above in a constraint, that will be replaced with a candidate string to check against in the constraint.

Working Backwards. First, fix the server's key k^* . Suppose we make up some constant $0 < s < 1$ and further make the assumption that after running $\mathcal{D}$ up to line 5 that every key k that satisfy all of the constraints in Γ has the property that,

$$\Pr\{F^{\mathcal{O}}(k, x) = F^{\mathcal{O}}(k^*, x) \mid x \leftarrow \mathcal{X}\} \geq s. \quad (1)$$

Later we will see how we must set s to get sufficient advantage. If a key k satisfies Equation (1), we say that it is *consistent*. Then if we choose any consistent key k and uniformly sample a challenge input, notated x^* , from a set $\mathcal{X} \setminus \mathcal{X}'$, where $\mathcal{X}'$ is any set at most an η fraction of the size of $\mathcal{X}$, then

$$\Pr[F^{\mathcal{O}}(k, x^*) = F^{\mathcal{O}}(k^*, x^*) \mid x^* \leftarrow \mathcal{X} \setminus \mathcal{X}'] \geq s - \eta,$$

noting that in the worst case all the keys k agree with k^* on the inputs from $\mathcal{X}'$, so we can only guarantee that the consistent keys k agree w.p. only $s - \eta$ when uniformly chosen from $\mathcal{X} \setminus \mathcal{X}'$. Specifically, if we let $\mathcal{X}'$ be the set of inputs over which interactive evaluations were made, then our attack can work even as x^* is drawn uniformly over $\mathcal{X} \setminus \mathcal{X}'$ instead of over $\mathcal{X}$.

This means that line 7 in $\mathcal{D}$ produces an attack with advantage $s - \eta$ as long as $|\mathcal{X}| \geq n/\eta$ and that the assumption we made holds, i.e., that by line 5 all the keys that satisfy all of the constraints in Γ satisfy Equation (1).

Resolving the Assumption. Now we will show that the analysis in lines 2-4 of $\mathcal{D}$ have at least a $1/2$ probability of satisfying the assumption that the remaining keys satisfy Equation (1) when n is set to $\ell(\lambda) \log_s(1/2)$, also still assuming that $|\mathcal{X}| \geq n/\eta$.

²The adversarial client will run through the computation again as if he was doing it.

First, call each key k that does not satisfy Equation (1) w.r.t. k^* an *inconsistent* key. Then we note that by definition, all the keys that do not satisfy Equation (1) each have at least a $1 - s$ probability of being thrown out each time a constraint on line 4 is added, since all keys k for which

$$F^{\mathcal{O}}(k, x_i) \neq F^{\mathcal{O}}(k^*, x_i)$$

will either be evaluable given the oracle queries Q_i but evaluate to $1 - y_i$, or not be evaluable given the provided oracle queries Q_i . In either case all of such keys will not satisfy the constraint added on that given iteration.

Of course, we hope that by line 5, all such inconsistent keys k are removed. We will see this by computing the expected number of inconsistent keys that remain after the for loop completes. Suppose there are m inconsistent keys and we assign an index to each. Let the random variable X_j^i denote whether inconsistent key k_j satisfies (resp. does not satisfy) the constraint added to Γ on line 4 on the i -th iteration of the for loop, where $X_j^i = 1$ when k_j satisfies the added constraint (resp. $X_j^i = 0$ when k_j does not satisfy the added constraint). Then by definition of an inconsistent key, we have that

$$\Pr[X_j^i = 1] < s \implies \mathbb{E}[X_j^i] < s.$$

Note that because k^* and inconsistent key k_j are fixed in advance, the only source of randomness for X_j^i is from the input of the client, which is uniform by definition. Therefore, two trials X_j^i and $X_j^{i'}$ for $i \neq i'$ are independent. Each key remains as a candidate w.r.t. Γ on line 5 only if it satisfies *every* constraint added by line 4, n in total. As such, the random variable determining whether inconsistent key k_j successfully satisfies each of the constraints is

$$\prod_i^n X_j^i.$$

Likewise, we would like to count the total number of inconsistent keys that pass all of the constraints, which is given by the random variable

$$\sum_j^m \prod_i^n X_j^i.$$

Note that because each x_i is drawn uniformly and interactive evaluations are unrelated, each new condition added by line 4 is an independent trial on X_j^i for any fixed inconsistent key k_j . Then combining the expectation above we find that

$$\mathbb{E}\left[\sum_j^m \prod_i^n X_j^i\right] = \sum_j^m \mathbb{E}\left[\prod_i^n X_j^i\right] = \sum_j^m \prod_i^n \mathbb{E}[X_j^i] < m \cdot s^n.$$

Then we note that there can only be a maximum of $2^{\ell(\lambda)}$ keys, let alone inconsistent keys. Thus, we find that by setting $n = (\ell(\lambda) + 1) \log_s(1/2)$, the expected number of remaining inconsistent keys is

$$m \cdot s^n = m \cdot s^{-(\ell(\lambda)+1) \log_s(2)} \leq \left(2^{\ell(\lambda)}\right) 2^{-(\ell(\lambda)+1)} = \frac{1}{2}.$$

Note that since it is discrete whether a key remains after $\mathcal{D}$ runs, we can convert this expectation into a $1/2$ probability that by line 5 in $\mathcal{D}$, all inconsistent keys are ruled out.

Conclusion. Recall that line 5-7 gives an attack with advantage of $s - \eta$ assuming a fact that occurs with probability $1/2$. As such, the total advantage is

$$\frac{1}{2}(s - \eta).$$

Thus, to produce an attack for an arbitrary constant allowable advantage, we set

$$2\varepsilon_i + \eta \leq s < 1.$$

□

3.4 Handling Incorrect Responses

Now we will extend our analysis to those OPRF schemes that are not perfectly correct. That is, we will consider OPRF protocols that, on uniformly chosen k and x , have a non-zero chance to output something other than $F^{\mathcal{O}}(k, x)$. Note that our previous attack does not work in the context of a possibly-incorrect protocol. Indeed, the attack given in Lemma 3 could rule out the server's key k^* after an incorrect interactive evaluation, which happens with ε_c probability each iteration. In short, we fix this issue by considering evaluations in λ -sized batches, and requiring keys to disagree with at least $1/3$ of the tests before being dismissed. For small enough ε_c , it is unlikely that k^* will be ruled out, but inconsistent keys will still fail many of the tests, as we will see.

Lemma 4 (Interactive Evaluation Bounds on OPRF Schemes with Bounded-Material-Query Servers). *Let $\Pi = (\text{Gen}, C, S, F)$ be an efficient OPRF scheme from a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles $\text{PK}\mathcal{O}$, which are independent from $\mathcal{O}$. Also require that F is a poly-sized circuit with only oracle access to $\mathcal{O}$ and S **cannot make oracle queries**. Let $\eta < 5/12 - 2^{-\Omega(\lambda)}$ be a constant. Say that the key length is bounded by $\ell(\lambda)$ and that $|\mathcal{X}| \geq (\lambda \cdot \ell(\lambda) + \lambda)/\eta$.*

Then it is not possible for Π to simultaneously satisfy correctness (Definition 2) and $(\lambda \cdot \ell(\lambda) + \lambda)$ -weak one-more security (Definition 4) with constant bounds – in place of $\text{negl}(\lambda)$ – $\varepsilon_c < 1/24$ and $\varepsilon_i < 1/4 \cdot (5/12 - \eta) - 2^{-\Omega(\lambda)}$ respectively.

Proof.

The Algorithm. As stated above, we will instead evaluate on many inputs at the same time for each iteration, only ruling out a key if it disagrees with some large portion, say $1/3$, of the evaluations. We present this modification in the following code block.

```

 $\mathcal{D}^{\mathcal{O}, \text{PSPACE}, \text{PKO}}(1^\lambda, x^*, y^*, (x_1, y_1, r_1, \tau_1), \dots, (x_{n\lambda}, y_{n\lambda}, r_{n\lambda}, \tau_{nc\lambda}))$ 
1:  $\Gamma \leftarrow \{ "k \in \text{Supp}(\text{Gen}(1^\lambda))" \}$ 
2: for  $i \in [n]$  :
3:   for  $j \in [\lambda]$  :
4:      $t \leftarrow (i-1) \cdot n + j$ 
5:     Record  $\mathcal{O}$  queries  $Q_j^i$  from  $y_t, -, - \leftarrow \langle C^{\mathcal{O}, \text{PSPACE}, \text{PKO}}(1^\lambda, x_t; r_t), \text{Dummy}(\tau_t) \rangle$ 
6:      $\Gamma \leftarrow \Gamma \cup \{ "$ 
7:        $q \leftarrow 0$ 
8:       for  $j \in [\lambda]$  :
9:          $t \leftarrow (i-1) \cdot n + j$ 
10:        if  $\text{Check}^{\text{PSPACE}}(F, Q_j^i, k, x_t, 1/3) \neq y_t$  :
11:           $q \leftarrow q + 1$ 
12:         $q < \lfloor \lambda/3 \rfloor$ 
13:       $" \}$ 
14:    $k \leftarrow \text{ConstraintSample}^{\text{PSPACE}}(\Gamma)$ 
15:   if  $k = \perp$  :
16:      $b' \leftarrow \$ \{0, 1\}$ 
17:     return  $b'$ 
18:   else :
19:      $b' \leftarrow F^{\mathcal{O}}(k, x^*)$ 
20:   return  $b' \oplus y^*$ 

```

Note that lines 6-13 are adding a constraint to Γ that happens to be more verbose (the constraint being defined in lines 7-12).

For the rest of the analysis we will leave out exponentially small probabilities, bringing them in at the very end as a very slight drain on advantage. Further, note that with at least $1/2$ probability, the server's key k^* sampled from $\text{Gen}(\lambda)$ is incorrect over at most a $2\varepsilon_c$ proportion of inputs, otherwise the protocol cannot be correct.

Not Ruling Out k^* . Suppose that k^* is a key which is correctly evaluated by the interactive protocol with probability at least ε'_c over all inputs. For sampling values in the following way

$$x \leftarrow \$ \mathcal{X}$$

$$\text{Record } \mathcal{O} \text{ queries } Q \text{ from } y, -, - \leftarrow \langle C^{\mathcal{O}, \text{PSPACE}, \text{PKO}}(1^\lambda, x), S^{\text{PSPACE}, \text{PKO}}(k^*) \rangle,$$

we would like to upper bound the following:

$$\Pr[\text{Check}^{\text{PSPACE}}(F, Q, k^*, x, 1/3) \neq y]. \quad (2)$$

First consider that there are two cases in which the check does not agree with y : outputs NO or is able to compute $F^{\mathcal{O}}(k^*, x)$ with at least $2/3$ accuracy and finds that it does not equal y .

Note that when insufficient random oracle queries are made by both the client and server to compute $F^{\mathcal{O}}(k^*, x)$ correctly with probability at least $2/3$, then the maximum probability that the interactive computation will be correct is less than $2/3$.

Now we will define some terms.

- Let δ be the probability that the output of **Check** is NO.

- Let e_A (resp. e_B) be the error probability of the interactive scheme w.r.t. $F^{\mathcal{O}}(k^*, x)$ when the output of **Check** is NO (resp. is not NO).

To start, note that $e_A + e_B < \varepsilon'_c$ by definition. As such, we find that $e_A > 2\delta/3$, where $2\delta/3$ is the total error due to not pinning the output of $F^{\mathcal{O}}(k^*, x)$ with at least $2/3$ probability. Then our goal given the above is to bound $\delta + e_B + \varepsilon_c$, which is the upper bound on the total probability that **Check** rejects the server's key k^* . In summary, we have

$$\begin{aligned} e_A + e_B &< \varepsilon'_c \\ \frac{2}{3}\delta &< e_A, \end{aligned}$$

which implies that

$$\delta + e_B + \varepsilon_c < \varepsilon_c + \frac{3}{2}\varepsilon'_c$$

by arithmetic we defer to Lemma 6 in Appendix C. Recall that we guarantee with $1/2$ probability that the error rate of k^* is $2\varepsilon_c = \varepsilon'_c$. As such, we can bound the probability given in Equation (2) by

$$1 - 4\varepsilon_c.$$

Suppose that we also require that $\varepsilon_c < 1/24$. This means that the expected number of evaluations that will be in error for server key k^* is at most $1/6$. As such, it is exponentially unlikely – by a Chernoff bound – that at least a $\lfloor \lambda/3 \rfloor$ fraction of the λ will be inconsistent. As such, throughout the entire execution of the program, the probability that k^* is ruled out is exponentially small, i.e., $2^{-\Omega(\lambda)}$.

A Note on Error. The notion of correctness error ε_c can be more broad than just the probability that the protocol outputs differently than F . In particular, the above arguments go through if we consider “error” to instead mean the probability that the protocol *looks like* it disagrees with the server's key k^* . In particular, the above arguments are robust against a protocol that may ask the client to make oracle queries that only guarantee correctness to ε_c , even though the protocol is able to somehow agree with F with error $\varepsilon'_c < \varepsilon_c$ probability. As long as ε_c is within our tolerances, we can be assured that k^* will only have an exponentially small probability of being crossed off.

We put this note here because this distinction about error will come up in the proof of Theorem 1, in which we will show that the protocol will potentially be able to create an error gap, but we will bound this gap.

Ruling out k . Now we analyze the probability that each inconsistent k is ruled out. Like last time, let $s < 1$ be some constant such that we would like to rule out all k that agree with k^* on fewer than an s fraction of points. Formally, by line 14 we would like there only to be candidate keys k such that

$$\Pr[F^{\mathcal{O}}(k^*, x) = F^{\mathcal{O}}(k, x) \mid x \leftarrow \$_{\mathcal{X}}] \geq s. \quad (3)$$

Note that to make sure that the interactive evaluation of k^* satisfies this condition, we require that $1 - 2\varepsilon_c \geq s$.

We will analyze the probability that a key k that does not satisfy Equation (3), i.e., an inconsistent key, is rejected by any particular test on line 10 of $\mathcal{D}$. Formally, for any k which does not satisfy Equation (3), and for the following sampling,

$$x \leftarrow \$_{\mathcal{X}}$$

$$\text{Record } \mathcal{O} \text{ queries } Q \text{ from } y, -, - \leftarrow \langle C^{\mathcal{O}, \text{PSPACE}, \text{PK}^{\mathcal{O}}}(1^\lambda, x), S^{\text{PSPACE}, \text{PK}^{\mathcal{O}}}(k^*) \rangle,$$

we would like to lower bound

$$\Pr[\text{Check}^{\text{PSPACE}}(F, Q, k, x, 1/3) \neq y]. \quad (4)$$

For ease of notation, let C be a random variable that outputs according to the distribution of $\text{Check}^{\text{PSPACE}}(F, Q, k, x, 1/3)$, and let Y be the random variable sampled according to the distribution that y as above comes from. Then we can rewrite Equation (4) as

$$\Pr[C \neq Y].$$

Note that by the $2\varepsilon_c$ correctness of k^* that

$$\Pr[F^{\mathcal{O}}(k^*, x) = Y] \geq 1 - 2\varepsilon_c. \quad (5)$$

Likewise, because Equation (3) does *not* hold for inconsistent keys k , we have that

$$\Pr[F^{\mathcal{O}}(k, x) \neq F^{\mathcal{O}}(k^*, x)] \geq 1 - s. \quad (6)$$

Lastly, we have two facts about the **Check** function:

$$\Pr[C \neq F^{\mathcal{O}}(k, x) \mid C = \text{NO}] = 1, \quad (7)$$

and

$$\Pr[C = F^{\mathcal{O}}(k, x) \mid C \neq \text{NO}] \geq \frac{2}{3}. \quad (8)$$

Using the above facts, we find that

$$\Pr[C = Y] < \frac{1}{3} + \frac{4}{3}\varepsilon_c + \frac{2}{3}s.$$

The arithmetic is deferred to Lemma 7 in Appendix C. If we want to maintain that $\Pr[C = Y] < 2/3$, using the fact from the last argument we made that $\varepsilon_c < 1/24$, then solving for s we can sustain $s \leq 5/12$. Recall that whether a given constraint rules out k is a Bernoulli trial, for which the above shows that the probability is at most $2/3$ for winning. As such, when we apply λ constraints, the probability that at least $\lceil 2/3 \cdot \lambda \rceil$ of the conditions are satisfied is at most $1/2$, since the above is the worst-case median of the binomial distribution.

This means that after each iteration of the for loop on line 2, each key not satisfying Equation (3) has at least a $1/2$ probability of being ruled out. Then by using the analysis from Section 3.3 to set n , we find that if we do $\ell(\lambda) + 1$ iterations (or $\lambda(\ell(\lambda) + 1)$ total interactive evaluations) the expected number of keys not satisfying Equation (3) remaining is $1/2$. Put another way, the probability that there are no inconsistent keys remaining is at least $1/2$.

Assuming that all keys satisfy Equation (3), we find that lines 14-20 can produce an attack with advantage $s \leq 5/12$. Recall that we have to factor in small-domain PRFs, for which sampling from the set $\mathcal{X} \setminus \mathcal{X}'$ for inputs $\mathcal{X}'$ that we have seen already materially effects the distribution, so our attack has advantage $5/12 - \eta$ if

$$|\mathcal{X}| \geq \lambda(\ell(\lambda) + 1)/\eta.$$

Further, we assumed that the server's key k^* has error probability at most $2\varepsilon_c$ – which occurs with probability at least $1/2$, and that no inconsistent keys appear with only probability $1/2$. Including this, the total advantage is then

$$\frac{1}{4} \left(\frac{5}{12} - \eta \right).$$

Conclusion. Recall that the reason we need to retain k^* is so that we can guarantee that at the end of the protocol, we still have at least one valid key to sample from. We also factor in the fact that $\mathcal{D}$ can fail with exponentially small probability, in which case a uniform guess is generated. This means that in total, we can break security when $\varepsilon_i < 1/4 \cdot (5/12 - \eta) - 2^{-\Omega(\lambda)}$ and $\varepsilon_c < 1/24$. $\square$

3.5 Handling Key Extension Queries

So far we have been forbidding the server from making any RO queries. In some sense, it is not possible to have the number of required evaluations to break security only scale with the key length, because given a short (say λ length) key, the server could use RO queries to extend the key arbitrarily. To take this into account, we simply consider our bound w.r.t. the number of such queries the server makes. In particular, the server can only make a bounded and polynomial number of RO queries, which can only have the effect of extending the key by some polynomial. We state our bound formally to handle such queries.

Lemma 5 (Interactive Query Bounds on Extension-Query Server OPRF Schemes). *Let $\Pi = (Gen, C, S, F)$ be an efficient OPRF scheme from a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles $PK\mathcal{O}$, which are independent from $\mathcal{O}$. Also require that F is a poly-sized circuit with only oracle access to $\mathcal{O}$, where $p_F(\lambda)$ is a polynomial bound on the number of oracle calls that F makes. Further, we require that S can only make some polynomial $p_S(\lambda)$ of $\mathcal{O}$ queries that fall in the category of “Extension queries”, defined as follows (fixing a constant $0 < s \leq 1$):*

- **Extension Query:** *w.r.t. the server’s key k , a query to $\mathcal{O}$ that is made by either F or S with at least s over uniformly chosen x .*

Let $\eta < 5/12 - 2^{-\Omega(\lambda)}$ be a constant. Say that the key length is bounded by $\ell(\lambda)$ and that

$$|\mathcal{X}| \geq (\lambda \cdot \ell(\lambda) + 4\lambda^2 \lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor + \lambda)/\eta.$$

Then it is not possible for Π to simultaneously satisfy correctness (Definition 2) and

$$(\lambda \cdot \ell(\lambda) + 4\lambda^2 \lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor + \lambda)$$

-weak one-more security (Definition 4) with constant bounds – in place of $\text{negl}(\lambda) - \varepsilon_c < 1/24$ and $\varepsilon_i < 1/4 \cdot (5/12 - \eta) - 2^{-\Omega(\lambda)}$ respectively.

Proof. We can reduce to Lemma 4 by having our attack instead imagine using a different underlying PRF than F . Suppose that our attack in Lemma 4 instead imagines that Π uses the following underlying PRF. Let $\mathcal{O}^Q$ agree with $\mathcal{O}$ except that for each $(q, o) \in Q$, $\mathcal{O}^Q(q) = o$.

$F'^{\mathcal{O}}((k, (q_1, o_1), \dots, (q_{\lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor}, o_{\lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor}), x)$ <pre> 1 : $Q \leftarrow \emptyset$ 2 : for $i \in [\lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor]$: 3 : if $(q_i, -) \notin Q$: 4 : $Q \leftarrow Q \cup \{(q_i, o_i)\}$ 5 : return $F^{\mathcal{O}^Q}(k, x)$ </pre>

Note that for any key k , there can only be a maximum of $\lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor$ extension queries. As such, for any key k and oracle $\mathcal{O}$ pair, there exists a key k' such that (1) for all $x \in \mathcal{X}$,

$$F^{\mathcal{O}}(k, x) = F'^{\mathcal{O}}(k', x)$$

and (2) the server need not make extension queries w.r.t. F' , since such queries would be handled by her long key k' . Note that because queries have λ -bit inputs and outputs, the keys k' of F' have length

$$\ell(\lambda) + 2\lambda \lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor.$$

Note that a reduction would see modifying $Gen(\lambda)$ so that it would simulate many instances of an interactive evaluation with a client who picks a uniform input, and then collect the most queries points into the key. To make sure that such a Gen does not leave out any extension query, except with exponentially

low probability, we run the modified Gen function as if s was halved. This means that the effective key k' is twice as long compared to if we could perfectly determine all the extension queries. Hence, we find that the keys have length

$$\ell(\lambda) + 4\lambda \lfloor (p_S(\lambda) + p_F(\lambda))/s \rfloor.$$

Some Other Considerations. Note that the random oracle is no longer independent of the key for a new OPRF scheme that uses F' as its underlying PRF; however, the proof of Lemma 4 only relies on the fact that given F' , a set of oracle queries Q , a candidate key k' , and an input x that Lemma 1 still holds with respect to any OPRF scheme as in the theorem statement w.r.t. F , even for keys k' containing RO input-output pairs. $\square$

3.6 Handling Non-Extension Queries and The General Case

For this section we will consider the other kind of queries not handled by the above: infrequent queries – defined above in Lemma 5 – which will complete the proof of our main result.

Theorem 1 (Interactive Evaluation Bounds on OPRF Schemes). *Let $\Pi = (\text{Gen}, C, S, F)$ be an efficient OPRF scheme from a random oracle $\mathcal{O}$, a PSPACE oracle, and arbitrary oracles PKO , which are independent from $\mathcal{O}$. Also require that F is a poly-sized circuit with only oracle access to $\mathcal{O}$. Say that the number of random oracle queries that F and S make during an execution is bounded by polynomials $p_F(\lambda)$ and $p_S(\lambda)$ respectively. Let $\eta < 5/12 - 2^{-\Omega(\lambda)}$ be a constant. Say that the key length is bounded by $\ell(\lambda)$ and that*

$$|\mathcal{X}| \geq (\lambda \cdot \ell(\lambda) + 36864\lambda^2 \cdot (p_S(\lambda) + p_F(\lambda))^2 + \lambda)/\eta.$$

Then it is not possible for Π to simultaneously satisfy correctness and one-time indistinguishability-based client privacy (Definitions 2 and 3) and

$$(\lambda \cdot \ell(\lambda) + 36864\lambda^2 \cdot (p_S(\lambda) + p_F(\lambda))^2 + \lambda)$$

-weak one-more security (Definition 4) with constant bounds (in place of $\text{negl}(\lambda)$) $\varepsilon_c \leq 1/96$, $\varepsilon_s \leq 1/9216$, and $\varepsilon_i \leq (5/12 - \eta)/4 - 2^{-\Omega(\lambda)}$ respectively.

Proof. Our goal is to show that RO queries that are not extension queries, which we call *infrequent* queries, that the server makes do not have a large impact over the correctness of the protocol over what the client predicts, due to such queries being so unlikely to occur and the fact that such queries cannot depend on the input of the client because of client privacy.³

In particular, this only affects the probability that the server's key may be rejected wrongly, since our attack may interpret having too little information to compute $F^{\mathcal{O}}(k^*, x)$ as the server's key being unlikely to be correct; however, we will show that the server's key should still be likely to be accepted, i.e. the amount more correct that the protocol can be over the estimate posed by our client and **Check** is some small constant, which will mean that the probability that the server's key is rejected in Lemma 4 is still exponentially small in λ , since our attack in Lemma 4 is robust against error of this kind. Thus, we only need to make sure that this error, added with ε_c , meets the bound given in Lemma 4, and by extension in Lemma 5, for error ε_c .

Best Possible Protocol Strategy. We start by going over the best strategy for the protocol for the purposes of producing correct output. Consider the end of an interactive evaluation, after the client and server have finished making all of their RO queries, in a set which we will call Q . Then the best possible strategy w.r.t correctness on the part of the protocol is to output what **Find** (Corollary 1) suggests, by definition. We are now trying to bound the probability output of **Find** between the protocol given both client and server RO queries, and the client, who is given only his own queries of course.

³We will call it client privacy for the extent of this proof, but note that we are referring to the weak notion of Definition 3 instead of the more standard simulation notion of Definition 5.

Perfect Client Privacy. We first consider the case of perfect client privacy, i.e. $\varepsilon_s = 0$. This means that the server's queries can only depend on her key k^* and the random oracle, but not at all on the client's input x to an interactive evaluation. Because we have already covered how to remove extension queries from F , consider only a list of queries that $F^\mathcal{O}(k^*, x)$ makes less often than w.p. s , w.r.t. a uniformly chosen x and over the randomness of the random oracle. By definition, each of these queries should be made by F with less than s probability, and further the server should be making such queries with less than s probability.

We consider two kinds of queries and give a bound on the amount they can contribute to the outcome of **Find**:

- **Infrequent Material Queries:** With respect to some client input x and random oracle $\mathcal{O}$, and of course the server's key k^* , these are infrequent random oracle queries that F makes to compute $F^\mathcal{O}(k^*, x)$. In particular, any such query that is not given to the client could help the protocol compute $F^\mathcal{O}(k^*, x)$ significantly better than the client could; however, each such query can be made by the server with no more than s probability. Since there can only be up to $p_F(\lambda)$ such queries at a given time, the total probability that any material query is made cannot exceed $s \cdot p_F(\lambda)$.
- **Infrequent Immaterial Queries:** These are infrequent queries that are not material, as above. We can bound the expected impact on the outcome of **Find** of infrequent immaterial queries directly. Since such queries will not be useful for directly computing $F^\mathcal{O}(k^*, x)$ directly for some given instance, they can only help toward resolving entropy in the probability analysis in **Find** for F . In particular, because infrequent queries are made by F w.p. at most s , their expected impact on the output of **Find** cannot exceed s . Since the server can only make up to $p_S(\lambda)$ queries, the expected impact of such queries cannot exceed $s \cdot p_S(\lambda)$.

Then we find that the total expected improvement of **Find** over what the client finds is no more than $s \cdot (p_S(\lambda) + p_F(\lambda))$. Formally,

$$\mathbb{E}[\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x)] \leq s \cdot (p_S(\lambda) + p_F(\lambda)),$$

where the expectation is over client RO queries Q_c , server RO queries Q_s , and uniformly chosen inputs x , and noting that we will pretend that **Find** only outputs the probability for notational convenience. Then we can use Markov's Inequality to find that

$$\Pr\left[\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x) \geq \frac{1}{96}\right] \leq 96s \cdot (p_S(\lambda) + p_F(\lambda)).$$

If we set $s = 1/(96^2 \cdot (p_S(\lambda) + p_F(\lambda)))$, then we can bound the above to $1/96$.

Then there are two cases.

- For the particular RO queries that the client and server make, $\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x) < \frac{1}{96}$, in which case the difference between the common output of **Find** from the client's perspective and that of the protocol's is small ($1/96$), which means that the effective error w.r.t. accidentally rejecting the server's key k^* in our attack only increases by $1/96$.
- In the event that the above bound does not hold, then in the worst case the server's key is rejected, but since this only happens w.p. at most $1/96$, this adds another $1/96$ probability for the server's key k^* to be rejected, per trial.

In total then, the probability for rejecting the true key due to the above comes to $1/48$. We will wait on drawing a conclusion about this before considering the next section.

Imperfect Client Privacy. In the setting where candidate OPRF protocols may not maintain perfect client privacy, note that Definition 3 can be interpreted as bounding the maximum change of the distribution of queries that the server makes. In particular, consider the distribution of oracle queries that the server

makes when the client evaluates over a uniformly chosen x . Then the total difference between that distribution and the distribution over queries that the server makes w.r.t. some fixed client input x' should be less than ε_s .

As such, we can set the uniform-input distribution as the “canonical” query distribution, using our prior rational in addition to pointing out that each query could have a shifted probability in sum by at most ε_s by definition of statistical distance. Note that the impact of infrequent immaterial queries are not effected by changing the likelihood that they are made by the server, as such we only need to consider infrequent material queries. For material queries, the likelihood that *any* infrequent material query can be made can only increase by ε_s .

As such, we find that the expectation is bounded by

$$\mathbb{E}[\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x)] \leq s \cdot (p_S(\lambda) + p_F(\lambda)) + \varepsilon_s.$$

Then again we use Markov’s Inequality to find that

$$\begin{aligned} \Pr \left[\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x) \geq \frac{1}{96} \right] \\ \leq 96s \cdot (p_S(\lambda) + p_F(\lambda)) + 96\varepsilon_s. \end{aligned}$$

If we require that $\varepsilon_s < 1/96^2$, and remembering we set $s = 1/(96^2 \cdot (p_S(\lambda) + p_F(\lambda)))$, we find that

$$\Pr \left[\mathbf{Find}(F, Q_c \cup Q_s, k^*, x) - \mathbf{Find}(F, Q_c, k^*, x) \geq \frac{1}{96} \right] < \frac{1}{48}.$$

Then the two cases above add probability $1/96$ and $1/48$ at most to rejecting the server’s key during each iteration of the attack. If we set $\varepsilon_c < 1/96$, then the total error w.r.t. key k^* failing a trial in Lemma 4 is bounded by

$$\frac{1}{96} + \frac{1}{96} + \frac{1}{48} = \frac{1}{24},$$

which is the allowed error for Lemma 4 and Lemma 5.

Conclusion. As such, we can use the above analysis to invoke Lemma 5 with

$$s = \frac{1}{96^2 \cdot (p_S(\lambda) + p_F(\lambda))} = \frac{1}{9216 \cdot (p_S(\lambda) + p_F(\lambda))},$$

$$\varepsilon_s < \frac{1}{96^2} = \frac{1}{9216}, \quad \varepsilon_c < \frac{1}{96}.$$

□

4 Acknowledgements

We would like to acknowledge Hastin Kapoor for his help with sanity checks related to statistics with respect to our lower bound and James Hulett for his help with statistics related to our upper bound.

References

- [ADDS21] Martin R. Albrecht, Alex Davidson, Amit Deo, and Nigel P. Smart. Round-optimal verifiable oblivious pseudorandom functions from ideal lattices. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 261–289. Springer, Cham, May 2021.
- [Bas22] Andrea Basso. Poster: A post-quantum oblivious PRF from isogenies. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 3327–3329. ACM Press, November 2022.
- [BC23] Dung Bui and Geoffroy Couteau. Improved private set intersection for sets with small entries. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 190–220. Springer, Cham, May 2023.
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- [BFH⁺20] Carsten Baum, Tore Kasper Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai. PESTO: Proactively secure distributed single sign-on, or how to trust a hacked server. pages 587–606, 2020.
- [BKW20] Dan Boneh, Dmitry Kogan, and Katharine Woo. Oblivious pseudorandom functions from isogenies. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 520–550. Springer, Cham, December 2020.
- [CHL22] Sílvia Casacuberta, Julia Hesse, and Anja Lehmann. SoK: Oblivious pseudorandom functions. pages 625–646, 2022.
- [CL17] Jan Camenisch and Anja Lehmann. Privacy-preserving user-auditable pseudonym systems. pages 269–284, 2017.
- [CM20] Melissa Chase and Peihan Miao. Private set intersection in the internet setting from lightweight oblivious PRF. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 34–63. Springer, Cham, August 2020.
- [DCZ⁺24] Shen Dong, Hongrui Cui, Kaiyi Zhang, Kang Yang, and Yu Yu. A simple post-quantum oblivious transfer protocol from mod-LWR. Cryptology ePrint Archive, Report 2024/1116, 2024.
- [DGH⁺20] Nico Döttling, Sanjam Garg, Mohammad Hajiabadi, Daniel Masny, and Daniel Wichs. Two-round oblivious transfer from CDH or LPN. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 768–797. Springer, Cham, May 2020.
- [DGS⁺18] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. Privacy pass: Bypassing internet challenges anonymously. *PoPETs*, 2018(3):164–180, July 2018.
- [DHL22] Poulami Das, Julia Hesse, and Anja Lehmann. DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 682–696. ACM Press, May / June 2022.
- [ECS⁺15] Adam Everspaugh, Rahul Chatterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart. The pythia PRF service. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 547–562. USENIX Association, August 2015.

- [FIPR05] Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold. Keyword search and oblivious pseudorandom functions. In Joe Kilian, editor, *TCC 2005*, volume 3378 of *LNCS*, pages 303–324. Springer, Berlin, Heidelberg, February 2005.
- [GJK⁺24] Yanqi Gu, Stanislaw Jarecki, Pawel Kedzior, Phillip Nazarian, and Jiayu Xu. Threshold PAKE with security against compromise of all servers. In *ASIACRYPT 2024, Part V*, *LNCS*, pages 66–100. Springer, Singapore, December 2024.
- [GPR⁺21] Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 395–425, Virtual Event, August 2021. Springer, Cham.
- [Haz15] Carmit Hazay. Oblivious polynomial evaluation and secure set-intersection from algebraic PRFs. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015, Part II*, volume 9015 of *LNCS*, pages 90–120. Springer, Berlin, Heidelberg, March 2015.
- [HIJ⁺21] Sharon Huang, Subodh Iyengar, Sundar Jeyaraman, Shiv Kushwah, Chen-Kuei Lee, Zutian Luo, Payman Mohassel, Ananth Raghunathan, Shaahid Shaikh, Yen-Chieh Sung, et al. Privatestats: De-identified authenticated logging at scale, 2021.
- [HL08] Carmit Hazay and Yehuda Lindell. Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 155–175. Springer, Berlin, Heidelberg, March 2008.
- [IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.
- [JKK14] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.
- [JKKX16] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). pages 276–291, 2016.
- [JKKX17] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17International Conference on Applied Cryptography and Network Security*, volume 10355 of *LNCS*, pages 39–58. Springer, Cham, July 2017.
- [JKX18] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.
- [JL10] Stanislaw Jarecki and Xiaomin Liu. Fast secure computation of set intersection. In Juan A. Garay and Roberto De Prisco, editors, *SCN 10*, volume 6280 of *LNCS*, pages 418–435. Springer, Berlin, Heidelberg, September 2010.
- [KKRT16] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious PRF with applications to private set intersection. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 818–829. ACM Press, October 2016.

- [KLS⁺17] Ágnes Kiss, Jian Liu, Thomas Schneider, N. Asokan, and Benny Pinkas. Private set intersection for unequal set sizes with mobile applications. *PoPETs*, 2017(4):177–197, October 2017.
- [KMM⁺25] Yashvanth Kondi, Ian McQuoid, Kelsey Melissaris, Claudio Orlandi, Lawrence Roy, and LaKyah Tyner. Onetwopake: Two-round strong asymmetric pake with ideal security. *Cryptology ePrint Archive*, 2025.
- [KMP⁺17] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1257–1272. ACM Press, October / November 2017.
- [Leh19] Anja Lehmann. ScrambleDB: Oblivious (chameleon) pseudonymization-as-a-service. *PoPETs*, 2019(3):289–309, July 2019.
- [MPR⁺20] Peihan Miao, Sarvar Patel, Mariana Raykova, Karn Seth, and Moti Yung. Two-sided malicious security for private intersection-sum with cardinality. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 3–33. Springer, Cham, August 2020.
- [MR19] Daniel Masny and Peter Rindal. Endemic oblivious transfer. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 309–326. ACM Press, November 2019.
- [NR04] Moni Naor and Omer Reingold. Number-theoretic constructions of efficient pseudo-random functions. *Journal of the ACM*, 51(2):231–262, March 2004.
- [PRTY19] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. SpOT-light: Lightweight private set intersection from sparse OT extension. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 401–431. Springer, Cham, August 2019.
- [PRTY20] Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. PSI from PaXoS: Fast, malicious private set intersection. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 739–767. Springer, Cham, May 2020.
- [PSSZ15] Benny Pinkas, Thomas Schneider, Gil Segev, and Michael Zohner. Phasing: Private set intersection using permutation-based hashing. In Jaeyeon Jung and Thorsten Holz, editors, *USENIX Security 2015*, pages 515–530. USENIX Association, August 2015.
- [PSZ14] Benny Pinkas, Thomas Schneider, and Michael Zohner. Faster private set intersection based on OT extension. In Kevin Fu and Jaeyeon Jung, editors, *USENIX Security 2014*, pages 797–812. USENIX Association, August 2014.
- [Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.
- [RS21] Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 901–930. Springer, Cham, October 2021.
- [TCR⁺22] Nirvan Tyagi, Sofia Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood. A fast and simple partially oblivious PRF, with applications. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 674–705. Springer, Cham, May / June 2022.

- [WLLM25] Xiaodong Wang, Zijie Lu, Bei Liang, and Shengzhe Meng. Unbalanced psi from client-independent relaxed oblivious prf. *Proceedings on Privacy Enhancing Technologies*, 2025.
- [YBH⁺25] Yibin Yang, Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, and Tal Rabin. Gold OPRF: Post-quantum oblivious power-residue PRF. In *2025 IEEE Symposium on Security and Privacy*, pages 259–278. IEEE Computer Society Press, May 2025.

Appendices

A Symbols

We provide a list of common symbols used in this work. The proofs involve many different objects, so we give a single place to reference symbols in case one forgets the general meaning of a symbol. This table does not need to be memorized, as the symbols should also be introduced as they come up, we simply provide this table for reference so that one does not need to search through pages to find where symbols are defined.

Notation 3 (Common Symbols).

- λ denotes a security parameter.
- $\mathcal{O}$, PSPACE , PKO denote the random oracle, PSPACE oracle, and set of public key oracles respectively.
- $\ell(\lambda)$, $p_S(\lambda)$, $p_F(\lambda)$ denote polynomial bounds on key length, server RO queries, and F RO queries respectively for a given OPRF scheme.
- n is a placeholder constant, usually denoting number of interactive queries.
- k^* denotes a “true” key; i.e. a key being used by a party – usually the server – for some specific instance of a game.
- k denotes a key in general, and a key different than k^* in context.
- $\mathcal{K}$ denotes a set of keys.
- s usually denotes some placeholder constant that will be associated with the strength of an attack.
- $\varepsilon_c, \varepsilon_i, \varepsilon_s$ denote constant bounds for correctness (Definition 2), indistinguishability (Definition 4), and client privacy (Definition 5) respectively for a candidate OPRF scheme.
- η denotes a constant that is the inverse multiple of the size of the domain to F w.r.t. the number of queries made. In particular, if the domain of F is exponentially large, then η can be thought of as negligible.
- Q denotes a set of queries.
- Γ, γ denotes a set of constraints and a single constraint respectively (Notation 2).
- $\mathcal{X}, \mathcal{Y}$ are the input and output space respectively of a given F or a hash family.
- x, x_i denote inputs to F .
- y, y_i denote outputs of F or from an interactive evaluation of a given OPRF scheme.
- x^*, y^* are challenge version of inputs and outputs respectively.
- $\mathcal{H}$ denotes an n -wise hash family for some n given in context.
- $h, h(\cdot)$ denotes a hash seed and hash execution respectively associated with a given hash family $\mathcal{H}$.

B PSPACE Algorithms

We present the constructions for the PSPACE algorithms that we introduced in Section 3.2.

Lemma 1 (Consistency Checker). *Let $F^\mathcal{O}(\cdot, \cdot)$ be a circuit with oracle access. There exists a PSPACE algorithm **Check** that on input (1) the description of F , (2) a set of queries $Q = \{(q, o) \mid \mathcal{O}(q) = o\}$, (3) inputs k and x , and (4) a parameter ε_c , returns one of the following:*

- *The unique value y s.t., given the oracle queries Q , it is the case that $F^\mathcal{O}(k, x) = y$ with probability at least $1 - \varepsilon_c$. (Note that this requires $\varepsilon_c < 1/2$ to guarantee uniqueness.)*
- *The symbol NO, if there is no such y .*

Proof.

Let n be the number of queries that F makes to $\mathcal{O}$, note that this will be fixed since F is a circuit. Then the PSPACE algorithm iterates through every possible output for every assignment of $\mathcal{O}$ outputs, giving preference for $\mathcal{O}$ outputs that are the same as prior queries or queries in Q . Then after counting up for every y , **Check** determines if the number of times y is output is greater than a $1 - \varepsilon_c$ fraction, then outputting y if so and NO if not. Note that **Check** will go through $2^{n\lambda}$ iterations, but will only need $n\lambda$ space to store that counter, otherwise nothing else is stored. $\square$

Lemma 2 (Constraint Sample). *Let Γ be a set of PSPACE constraints over the set $\{0, 1\}^{\ell(\lambda)}$ for some implicit $\ell(\lambda)$. Then there exists a PSPACE algorithm **ConstraintSample** that returns one of the following:*

- *The smallest string k (when interpreted as an $\ell(\lambda)$ -bit integer) that satisfies all of the constraints in Γ .*
- $\perp$ *if no such $k \in \{0, 1\}^{\ell(\lambda)}$ exists.*

Proof. We construct a PSPACE algorithm for the above.

ConstraintSample ^{PSPACE} (Γ)	
1 :	for $k \in [2^{\ell(\lambda)}]$:
2 :	$c \leftarrow 0$
3 :	for $\gamma \in \Gamma$:
4 :	if $\neg \gamma^{\text{PSPACE}}(k)$:
5 :	$c \leftarrow 1$
6 :	if $c = 0$:
7 :	return k
8 :	return $\perp$

Recall that PSPACE is closed under composition, so the above algorithm can be formulated as a single call to a PSPACE oracle, even though γ may require PSPACE to evaluate. $\square$

C Deferred Arithmetic

In this section we present the deferred arithmetic from Lemma 4.

Lemma 6 (Deferred Arithmetic 1). *If*

$$e_A + e_B < \varepsilon'_c$$

$$\frac{2}{3}\delta < e_A,$$

then

$$\delta + e_B + \varepsilon_c < \frac{3}{2}\varepsilon'_c.$$

Proof.

$$\begin{aligned}
&\implies \delta < \frac{3}{2} \cdot e_A \\
&\implies \frac{2}{3}\delta + e_B < \varepsilon'_c \\
&\implies \varepsilon'_c - \frac{2}{3}\delta > e_B \\
&\implies \delta + e_B + \varepsilon_c < \frac{1}{3}\delta + \varepsilon_c + \varepsilon'_c \\
&\qquad\qquad\qquad < \frac{1}{2}e_A + \varepsilon_c + \varepsilon'_c \\
&\qquad\qquad\qquad < \frac{1}{2}\varepsilon'_c + \varepsilon_c + \varepsilon'_c \\
&\qquad\qquad\qquad = \varepsilon_c + \frac{3}{2}\varepsilon'_c.
\end{aligned}$$

□

Lemma 7 (Deferred Arithmetic 2). *If*

$$\Pr[F^\mathcal{O}(k^*, x) = Y] \geq 1 - 2\varepsilon_c, \quad (5)$$

$$\Pr[F^\mathcal{O}(k, x) \neq F^\mathcal{O}(k^*, x)] \geq 1 - s, \quad (6)$$

$$\Pr[C \neq F^\mathcal{O}(k, x) \mid C = \text{NO}] = 1, \quad (7)$$

and

$$\Pr[C = F^\mathcal{O}(k, x) \mid C \neq \text{NO}] \geq \frac{2}{3}, \quad (8)$$

then

$$\Pr[C = Y] \geq \frac{1}{3} + \frac{4}{3}\varepsilon_c + \frac{2}{3}s.$$

Proof.

$$\begin{aligned}
&\Pr[C \neq Y] \\
&\geq \Pr[C \neq Y \mid F^\mathcal{O}(k^*, x) = Y] \cdot \Pr[F^\mathcal{O}(k^*, x) = Y] \\
&= \Pr[C \neq F^\mathcal{O}(k^*, x)] \cdot \Pr[F^\mathcal{O}(k^*, x) = Y]
\end{aligned}$$

then by Equation (5)

$$\begin{aligned}
&\geq (1 - 2\varepsilon_c) \Pr[C \neq F^\mathcal{O}(k^*, x)] \\
&= (1 - 2\varepsilon_c) \Pr[C \neq F^\mathcal{O}(k^*, x) \mid C = \text{NO}] \cdot \Pr[C = \text{NO}] \\
&\quad + (1 - 2\varepsilon_c) \Pr[C \neq F^\mathcal{O}(k^*, x) \mid C \neq \text{NO}] \cdot \Pr[C \neq \text{NO}]
\end{aligned}$$

then by Equation (7)

$$\begin{aligned}
&= (1 - 2\varepsilon_c) \Pr[C = \text{NO}] \\
&\quad + (1 - 2\varepsilon_c) \Pr[C \neq F^\mathcal{O}(k^*, x) \mid C \neq \text{NO}] \cdot \Pr[C \neq \text{NO}] \\
&\geq (1 - 2\varepsilon_c) \Pr[C = \text{NO}] \\
&\quad + (1 - 2\varepsilon_c) \Pr[C \neq F^\mathcal{O}(k^*, x) \mid C = F^\mathcal{O}(k, x) \wedge C \neq \text{NO}] \\
&\quad \cdot \Pr[C = F^\mathcal{O}(k, x) \mid C \neq \text{NO}] \cdot \Pr[C \neq \text{NO}]
\end{aligned}$$

then by Equation (7)

$$\begin{aligned}
&= (1 - 2\varepsilon_c) \Pr[C = \text{NO}] \\
&\quad + (1 - 2\varepsilon_c) \Pr[F^\mathcal{O}(k, x) \neq F^\mathcal{O}(k^*, x)] \\
&\quad \cdot \Pr[C = F^\mathcal{O}(k, x) \mid C \neq \text{NO}] \cdot \Pr[C \neq \text{NO}]
\end{aligned}$$

then by Equations (6) and (8)

$$\begin{aligned}
&\geq (1 - 2\varepsilon_c) \Pr[C = \text{NO}] \\
&\quad + (1 - 2\varepsilon_c)(1 - s) \frac{2}{3} \cdot \Pr[C \neq \text{NO}] \\
&\geq (1 - 2\varepsilon_c)(1 - s) \frac{2}{3} \geq \frac{2}{3} - \frac{4}{3}\varepsilon_c - \frac{2}{3}s.
\end{aligned}$$

Then we find that

$$\Pr[C = Y] = 1 - \Pr[C \neq Y] < \frac{1}{3} + \frac{4}{3}\varepsilon_c + \frac{2}{3}s.$$

□

D Standard OPRF Security Definitions

The following definitions are standard and used among the literature. We chose weaker definitions to prove our lower bound against for the purpose of capturing weaker or non-standard definitions, but the definitions of OPRF security we presented in Section 2 do capture all of the following definitions, which is to say that the following definitions are also ruled out by our lower bound.

Simulation-Based Security. The following two definitions state the standard notions of simulation security for OPRFs, where a semi-honest adversary corrupts the server and client respectively. Both definitions are also adapted from [FIPR05]. In particular, these definition are the natural result of security from plugging a PRF into a semi-honest MPC scheme.

Definition 5 (Semi-Honest Oblivious PRF Client Privacy). *An OPRF scheme Π is **client private** if for all polynomials p , there exists a PPT simulator $\mathcal{S}$ such that for all stateful PPT adversaries $\mathcal{A}$, for all keys $k \in \text{Supp}(\text{Gen}(1^\lambda))$, and for the following game,*

$\text{Real}_{\Pi, p, \mathcal{A}, k}^{\text{c-priv}}(\lambda)$	
1:	$x_1 \leftarrow \mathcal{A}(1^\lambda)$
2:	for $i \in [p(\lambda)]$:
3:	$r_i \leftarrow_{\$} \{0, 1\}^*$
4:	$y_i, \neg, \tau_i \leftarrow \langle C(1^\lambda, x_i), S(k; r_i) \rangle$
5:	$x_{i+1} \leftarrow \mathcal{A}(y_i)$
6:	return $(k, (r_1, \tau_1), \dots, (r_{p(\lambda)}, \tau_{p(\lambda)}))$

The following indistinguishability holds:

$$\{\text{Real}_{\Pi, p, \mathcal{A}, k}^{\text{c-priv}}(\lambda)\} \approx \{\mathcal{S}(k)\}.$$

Definition 6 (n -Semi-Honest Oblivious PRF Server Privacy). *An OPRF scheme Π is **n -server private** if there exists a stateful PPT simulator $\mathcal{S}$, such that for all stateful PPT adversaries $\mathcal{A}$, for all keys $k \in \text{Supp}(\text{Gen}(\lambda))$, and for the following game*

$\text{Game}_{\Pi, \mathcal{A}, n, b}^{\text{om}}(\lambda)$	$\text{Game}_{\Pi, \mathcal{A}, n, b}^{\text{am}}(\lambda)$
1: $k \leftarrow \text{Gen}(1^\lambda)$	1: $k \leftarrow \text{Gen}(1^\lambda)$
2: $\mathcal{X}' \leftarrow \emptyset$	2: $\mathcal{X}' \leftarrow \emptyset$
3: $x_1 \leftarrow \mathcal{A}(1^\lambda)$	3: $x_1 \leftarrow \mathcal{A}(1^\lambda)$
4: for $i \in [n]$:	4: for $i \in [n]$:
5: $\mathcal{X}' \leftarrow \mathcal{X}' \cup \{x_i\}$	5: $\mathcal{X}' \leftarrow \mathcal{X}' \cup \{x_i\}$
6: $r_i \leftarrow_{\$} \{0, 1\}^*$	6: $r_i \leftarrow_{\$} \{0, 1\}^*$
7: $y_i, -, \tau_i \leftarrow \langle C(1^\lambda, x_i; r_i), S(k) \rangle$	7: $y_i, -, \tau_i \leftarrow \langle C(1^\lambda, x_i; r_i), S(k) \rangle$
8: $x_{i+1} \leftarrow \mathcal{A}(y_i, r_i, \tau_i)$	8: $x_{i+1} \leftarrow \mathcal{A}(y_i, r_i, \tau_i)$
9: $x^* \leftarrow \mathcal{A}(y_n, r_n, \tau_n)$	9: if b :
10: if $x^* \in \mathcal{X}'$:	10: return $\mathcal{A}^{f_{-\mathcal{X}'}(\cdot)}(y_n, r_n, \tau_n)$
11: return $\perp$	11: else :
12: $y_0^* \leftarrow F(k, x^*)$	12: return $\mathcal{A}^{F_{-\mathcal{X}'}(k, \cdot)}(y_n, r_n, \tau_n)$
13: $y_1^* \leftarrow_{\$} \mathcal{Y}$	
14: return $\mathcal{A}(y_b^*)$	

Figure 2: Above are the games for n -one-more (Definition 7) and n -any-more OPRF (Definition 8) security on the left and right, respectively. For the any-more security game, $f_{-\mathcal{X}'}(\cdot)$ and $F_{-\mathcal{X}'}(k, \cdot)$ return $\perp$ when queried on $x \in \mathcal{X}'$.

$\text{Real}_{\Pi, \mathcal{A}, n, k}^{\text{s-priv}}(\lambda)$	$\text{Ideal}_{\Pi, \mathcal{A}, \mathcal{S}, n, k}^{\text{s-priv}}(\lambda)$
1: $x_1 \leftarrow \mathcal{A}(1^\lambda)$	1: $x_1 \leftarrow \mathcal{S}^{\mathcal{A}}(1^\lambda)$
2: for $i \in [n]$:	2: for $i \in [n]$:
3: $r_i \leftarrow_{\$} \{0, 1\}^*$	3: $y_i \leftarrow F(k, x_i)$
4: $y_i, -, \tau_i \leftarrow \langle C(1^\lambda, x_i; r_i), S(k) \rangle$	4: $x_{i+1} \leftarrow \mathcal{S}^{\mathcal{A}}(y_i)$
5: $x_{i+1} \leftarrow \mathcal{A}(y_i, r_i, \tau_i)$	5: return $\mathcal{S}^{\mathcal{A}}(\perp)$
6: return $\mathcal{A}(\perp)$	

Then

$$\{\text{Real}_{\Pi, \mathcal{A}, n, k}^{\text{s-priv}}(\lambda)\} \approx \{\text{Ideal}_{\Pi, \mathcal{A}, \mathcal{S}, n, k}^{\text{s-priv}}(\lambda)\}.$$

Standard Weaker Security Notions. We adapt a common weakening of server security from [Leh19], in which a corrupted client cannot distinguish an unseen query of his choice from randomness, even after making n queries. Note that the original notion sees the adversary being able to making any unbounded polynomial number of queries, while the following definition bounds the queries by n .

Definition 7 (n -Oblivious PRF One-More Security). *An OPRF scheme Π is **one-more secure** if for all stateful PPT adversaries $\mathcal{A}$, and for the game on the left in Figure 2. Then*

$$|\Pr[\text{Game}_{\Pi, \mathcal{A}, n, 0}^{\text{om}}(\lambda) = 1] - \Pr[\text{Game}_{\Pi, \mathcal{A}, n, 1}^{\text{om}}(\lambda) = 1]| < \text{negl}(\lambda).$$

Note that the unbounded-evaluation version of Definition 7 implies security against an adversary who also gets to suggest an unbounded number of challenge F queries. Definition 7 is shown in favor since dealing with a single query to F is easier to show results for; however, in the bounded evaluation case, the

two notions are no longer equivalent. As such, we propose a natural strengthening of Definition 7 in which the adversary can suggest any polynomial number of challenge queries to F after making no more than n interactive evaluations.

Definition 8 (*n*-Oblivious PRF Any-More Security). *An OPRF scheme Π is **any-more secure** if for all stateful PPT adversaries $\mathcal{A}$, for random function $f : \mathcal{X} \rightarrow \mathcal{Y}$, and for the game on the right in Figure 2. Then*

$$|\Pr[\text{Game}_{\Pi, \mathcal{A}, n, 0}^{\text{am}}(\lambda) = 1] - \Pr[\text{Game}_{\Pi, \mathcal{A}, n, 1}^{\text{am}}(\lambda) = 1]| < \text{negl}(\lambda).$$

Relation to Unpredictability. There's a more common, weaker notion called one-more unpredictability from [JL10, ECS⁺15, Leh19], which is similar to Definitions 7 and 8, except that the adversary needs to guess the challenge query output instead of distinguishing from uniform. Our results relate to PRFs that output to single bits, for which the notions of indistinguishability and unpredictability are the same. Further, any unpredictable function F can be transformed into a PRF by passing the output through a random oracle.

E Constructions

Now we give some upper bounds for OPRF schemes for PRFs of the form $F^{\mathcal{O}} : \{0, 1\}^{(n+1)\lambda} \times \{0, 1\}^{\lambda} \rightarrow \{0, 1\}^{\lambda}$, with perfect correctness, exponentially secure client privacy, and n -any-more security.

We start by showing how to get n -one-more security from just an $(n+1)$ -wise independent hash function. In particular, the following construction does not require the ROM.

Lemma 8 (*(n+1)-wise Independent Hash Family as OPRF*). *The OPRF scheme where $F(h, x) = h(x)$ for $h \leftarrow \mathcal{H}$ with interactive protocol directly evaluating F via MPC is perfectly correct, exponentially secure client private, and exponentially n -one-more secure (Definitions 2, 6 and 7) in the MPC hybrid model and if $\mathcal{H}$ is an $(n+1)$ -wise independent hash family.*

Proof. The above lemma follows immediate from the indistinguishability of the following game for n -wise independent hash families.

$\text{Game}_{\mathcal{H}, \mathcal{A}, n, b}^{\text{uh}}(\perp)$	
1 :	$h \leftarrow \mathcal{H}$
2 :	$x_1 \leftarrow \mathcal{A}(\perp)$
3 :	for $i \in [n]$:
4 :	if b :
5 :	$y_i \leftarrow \mathcal{Y}$
6 :	else :
7 :	$y_i \leftarrow h(x_i)$
8 :	$x_{i+1} \leftarrow \mathcal{A}(y_i)$
9 :	return $\mathcal{A}(\perp)$

The output of the above games are indistinguishable between $b = 0, 1$ when we consider partially setting of inputs $x_1, \dots, x_{i-1} \in \mathcal{X}$ and outputs $y_1, \dots, y_{i-1} \in \mathcal{Y}$, in which we see that when

$$h(x_1) = y_1 \wedge \dots \wedge h(x_{i-1}) = y_{i-1},$$

then for any choice of $x_i \in \mathcal{X}$, we find that $h(y_i)$ is uniform. Viewing this inductively, having seen any prior outcomes, the next output of the hash h will still be uniform.

Then we notice that the above game is essentially what is required for n -one-more security when using an $(n+1)$ -wise independent hash function, which completes the proof. $\square$

We can upgrade the security of a $(n + 1)$ -wise independent hash family by running the output through a random oracle. By doing this, as long as the outputs of $h(\cdot)$ cannot be guessed or forced to collide, we can use the random oracle to make many queries to h look uniformly random, even if h does not guarantee the independence of many queries.

Construction 1 (Candidate OPRF). *Let $F^{\mathcal{O}}(h, x) = \mathcal{O}(\mathcal{O}_{\frac{1}{2}}(x) || h(x))$, where $h \leftarrow_{\$} \mathcal{H}$ for $(n + 1)$ -wise independent hash family $\mathcal{H}$ and $h : \{0, 1\}^{\lambda} \rightarrow \{0, 1\}^{\lambda/2}$, and $\mathcal{O}_{\frac{1}{2}}$ outputs the first half of the output from $\mathcal{O}$. The interactive portion of the OPRF scheme is handled by MPC, with the client making every oracle call.*

Theorem 2 (Construction 1 is n -Any-More Secure). *Protocol Π described in Construction 1 is perfectly correct, exponentially secure client private, and exponentially n -any-more secure (Definitions 2, 6 and 8) in the MPC hybrid model and if $\mathcal{H}$ is an $(n + 1)$ -wise independent hash family.*

Proof. First note that correctness and client privacy are maintained from Lemma 8, since the server has exactly the same view as the protocol given in Lemma 8.

Any-More Security. Consider each of the unique queries that the adversary makes after the interactive phase. Because $F^{\mathcal{O}}(h, x) = \mathcal{O}(\mathcal{O}_{\frac{1}{2}}(x) || h(x))$, we have that the probability that the adversary can force a collision on inputs is exponentially small, since at the very least the adversary would have to find $\mathcal{O}_{\frac{1}{2}}(x) = \mathcal{O}_{\frac{1}{2}}(x')$ for $x \neq x'$.

For the sake of contradiction, suppose that the adversary is able to distinguish in the game in Definition 8 with advantage more than any exponential. Notice that the only way for the adversary to distinguish between the two games is then to make a query $\mathcal{O}_{\frac{1}{2}}(x) || h(x)$ to $\mathcal{O}$ for some x and find that the output of $\mathcal{O}$ does/does not agree with what was given by the challenger. Thus, the adversary must be making an RO query $\mathcal{O}_{\frac{1}{2}}(x_i) || h(x_i)$ for some challenge query x_i that they made. Call this a critical query for x_i .

Indeed, if the adversary happens to make at least one critical query among the polynomially many queries, we can simply guess which one will be the first query, and w.r.t. to which x_i . Because all other queries are unrelated to the prior challenges x_j for $j \neq i$, we can construct an adversary that ignores queries to $F^{\mathcal{O}}(k, \cdot)$, returning a uniform output except for the i -th unique query, but otherwise answers oracle queries properly. Then we note that because the output of $F^{\mathcal{O}}(k, \cdot)$ is the output of $\mathcal{O}$ on one of these critical queries, uniformly sampling such outputs for non-critical queries perfectly matches the output distribution of F as long as the adversary has not made a critical query yet. In particular, we do not care about what the adversary's behavior is after making the first critical query.

As such, the above reduction will produce an attack with a polynomial advantage loss w.r.t. any hypothetical attack against the n -any-more security of Construction 1, which still is a super-exponential advantage against the protocol of Lemma 8, which is exponentially secure. $\square$

Corollary 2 (Construction 1 is n -Server Private). *Protocol Π described in Construction 1 is perfectly correct, exponentially secure client private, and exponentially n -server private (Definitions 2 and 6) in the MPC hybrid and programmable random oracle models and if $\mathcal{H}$ is an $(n + 1)$ -wise independent hash family.*